

Amortized Multi-Site Activation Patching via Cache-Replay Restoration in Persistent Computational Graphs

Abdallah Khemais

Abstract

Activation patching localizes which internal computations of a neural network are causally responsible for its behaviour, but a real causal-tracing study sweeps K sites, and tape-based frameworks (PyTorch, JAX) pay a full forward pass per site. NEURODSL’s persistent computational graph makes a single patch cost only its downstream impact subgraph; we show that a full sweep hides a second cost – restoring the corrupted baseline between sites – and that this cost can be eliminated entirely: the baseline is already cached, so restoration is a copy, not a recomputation. We introduce `restore_from_cache!` and `sweep_patch_sites!`, prove them exactly equivalent to recomputation-based restoration (exact equality, not tolerance-bounded), and measure a **36.1%** reduction in total sweep cost on an 8-layer CPU Transformer. The same mechanism extends, unmodified, to simultaneous multi-site patches (commutative for independent sites, cost bounded by the union of their cones) and to a greedy circuit search with an overlap-aware restoration strategy. On a model trained on the induction task, the unmodified search recovers a set of attention heads reaching 99.94% of the causal effect; its first two heads are independently confirmed by their attention patterns, and backward pruning shows four layer-1 heads alone carry the circuit. At scale (up to $\sim 1.82\text{B}$ parameters, GPU, clock-controlled), all correctness invariants hold exactly, and – after batching restoration into a single kernel launch and removing an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ invalidation-bookkeeping cost, both verified byte-identical – the amortization gain reaches 37–39%, above the CPU figure. Cross-validation against PyTorch on identical weights (parity 1.23×10^{-6}) shows a crossover, not a uniform speedup: NEURODSL trails at the shallowest site and leads the naive hook by up to $10\times$ at the deepest. We further prove a structural ceiling on this crossover: any exhaustive, depth-uniform sweep’s aggregate speedup converges to exactly $2\times$ regardless of scale, width, or head count – ruling out scale alone as a route to closing the ACDC/AtP* fidelity-cost gap, while the same analysis shows the mechanism makes AtP*’s own top- k verification step nearly free. Finally, the same persistence makes mid-training structural surgery cheap: a grafted layer outperforms an equal-budget fresh control (1.5647 vs. 1.5995 nats/char), while three pre-registered criteria for diagnosis-directed graft placement all fail – a clean negative result that the cheap diagnose-graft-rediagnose cycle made testable in minutes.

1 Introduction

Mechanistic interpretability seeks to explain a neural network’s behaviour by identifying which internal computations are causally responsible for it. **Activation patching** (also called causal tracing) [4, 5] is the workhorse technique for this: run the model once on a “clean” input and once on a “corrupted” input, then copy a single internal activation from the clean run into the corrupted run and observe how much of the output moves back towards the clean behaviour. The unit of a real study is not one patch but a **sweep**: repeating this at every layer (and often every attention head or token position) to localize where the causally relevant computation lives. Tooling such as TransformerLens [6] has made this sweep protocol a standard part of the mechanistic-interpretability toolkit.

In tape-based frameworks, the cost of a sweep is K full forward passes: PyTorch [2] and JAX [3] rebuild the computational graph from nothing on every call, so patching one node and observing the output costs exactly the same as patching any other, regardless of how much computation actually lies downstream of the patched site.

NEURODSL [1] maintains a persistent, mutable computational graph in which every node tracks whether its cached value is still valid. Writing a node invalidates only its true downstream dependents (its impact

subgraph), and the next demand-driven evaluation recomputes exactly that subgraph – a mechanism proven to cost $\mathcal{O}(|\mathcal{V}_\theta|)$, independent of total graph size, for parameter updates during training [1]. Activation patching is, mechanically, the same operation (a node write followed by a demand), so a single patch inherits the same bound for free.

A full sweep, however, is not just K independent cheap patches: each site must be tested from a common corrupted baseline, so the graph must be restored between sites. If restoration is done the same way a patch is applied – write the corrupted value back, then let the graph re-invalidate and recompute – it pays exactly the same recomputation cost as the patch it is undoing. Summed across a sweep, this restoration cost is of the same order as the sweep’s own measurement cost, eating a substantial fraction of the saving a single cheap patch promised. We show that this is avoidable: the corrupted baseline was already computed once, in full, before the first patch. Restoring to it is not new information – it requires no computation, only a direct copy of values already sitting in a cache.

We make twelve contributions:

1. **patch_node!** (§3), a value-injection primitive built on NEURODSL’s existing invalidation traversal, together with the precise reason the graph’s generic value-update primitive (**set!**) is unsuitable for this purpose.
2. **restore_from_cache!** and **sweep_patch_sites!** (§4), which restore a patched site’s downstream cone by direct cache replay instead of recomputation, and an amortized sweep orchestrator built on top of it – a capability specific to a persistent graph, where every node is individually addressable and its validity state explicit; a framework that rebuilds its graph from scratch has no equivalent notion of “this node, restored, without recomputing it.”
3. **patch_nodes!** and **restore_nodes_from_cache!** (§5), extending the same mechanism to simultaneous multi-site patches: commutative for independent sites, with cost bounded by the union of their cones rather than the sum, using no new graph-level primitive.
4. **greedy_patch_search!** and **_adaptive_restore!** (§6), a greedy search over candidate sites that restores cheaply by cache when provably safe to do so and falls back to exact recomputation whenever a candidate’s cone overlaps an already-selected site’s – a case where cache-based restoration would otherwise silently discard the selected site’s still-active contribution.
5. A structural-signal analysis (§9) asking whether a site’s downstream cone size – computable from **_downstream_nodes** without ever running a patched forward pass – predicts its causal importance: it does across depths, closely tracking both cost and recovery, but among sibling sites that share a depth, where cone size is structurally invariant, it carries no information at all, and causal importance still requires direct measurement.
6. A validation of **greedy_patch_search!** against a documented ground-truth circuit (§10): trained on the induction task [7], the same search that operates blindly over cost and recovery recovers a small set of attention heads reaching 99.94% of the causal effect, and the two heads it finds first are independently confirmed, by inspecting attention patterns the search never examined, to attend almost entirely to the exact token position the induction algorithm must copy from.
7. **backward_prune!** (§10), applied to the trajectory above: greedy search only ever adds sites, so having found five, we ask whether any has become redundant once the others are present. Doing so safely requires more care than removing sibling sites (§5), since three of the five lie strictly upstream of a fourth via the residual stream; **backward_prune!** avoids the resulting silent-invalidation hazard by never undoing a site in place, testing every candidate subset from a full reset instead. It removes the last-added, layer-2 site with no measurable loss, showing the four layer-1 heads alone already constitute the discovered circuit’s causal core.
8. A correctness-first empirical evaluation (§7–§9) on a small Transformer: per-patch cost and recovery as a function of layer depth (cross-checked by a marginal-cost decomposition that sums to exactly the independently-measured full-forward cost), a 36.1% reduction in total sweep cost from cache-replay restoration, a joint-vs-additive recovery comparison for two composed sites, and a 4-step greedy search

trajectory whose final recovery is independently reproducible, with every speed number preceded by an exact-equality correctness check.

9. A scale check the CPU benchmark alone cannot provide, and a fix for what it finds (§12): on GPU, up to ~ 1.82 -billion parameters, all correctness invariants hold exactly, unchanged by scale or device. An initial timed run understated the amortization gain; tracing the shortfall surfaced two independent causes rather than one – a GPU power-management state that declined to boost clocks for this workload (removed by locking the clock, with the machine owner’s permission), and an unrelated $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ invalidation-bookkeeping cost in `patch_node!` that a cached consumers index removes entirely (§12.5). A new batched restoration kernel, `restore_from_cache_batched!` (verified byte-identical to the existing mechanism), combined with both fixes, lifts the measured gain to 37–39% at every width re-tested ($\sim 0.05\text{B}$, $\sim 0.81\text{B}$, $\sim 1.82\text{B}$ parameters), *above* the 36.1% CPU figure. A roofline-style explanation offered for the original, smaller shortfall does not survive this correction and is retracted in §12.8: the gap was substantially an engineering artifact, not a hardware limit.
10. A quantitative cross-framework validation (§11.1): weights exported to a PyTorch re-implementation of the same architecture, parity verified to 1.23×10^{-6} before any timing claim. Across 20 attention-head sites, NEURODSL trails a naive PyTorch hook at the shallowest site, where its cone is close to the entire remaining graph, and leads it by up to $10\times$ at the deepest – a crossover that makes the structural argument of §11 concrete rather than a uniform speedup claim.
11. A proof, not just a measurement, that an exhaustive depth-uniform sweep’s aggregate speedup is capped at exactly $2\times$ as depth grows, independent of width or head count (Theorem 1, §11.2) – closing off, by construction, the possibility that scale alone lets NEURODSL close the fidelity/cost gap between exact circuit discovery (ACDC [10]) and fast approximate attribution (AtP* [11]) in the exhaustive-coverage regime; the same closed form shows the mechanism instead makes AtP*’s own recommended top- k exact-verification step, which touches only deep, late sites, nearly free.
12. `insert_block!` and `graft_shadow_block!` (§13), extending the same persistence from patching values to mutating structure: a fifth layer grafted mid-training into a real trained language model, continued for the same total budget as a fresh same-sized control, reaches a lower validation loss (1.5647 vs. 1.5995 nats/char). Testing whether a causal diagnosis can *direct* where such a graft is placed, rather than only describe the network afterward, three pre-registered criteria all fail on the task built to test them (§13.2) – a negative result the same cheap mechanism made testable in minutes rather than left unasked.

A companion paper exploits `insert_block!` for a third purpose beyond patching and directed capacity placement – growing a network’s depth during training – and is reported separately, since it shares only the mechanism with this paper, not its method or question.

All experiments use NEURODSL’s existing, already-tested `LlamaModel` (`src/layers.jl`); no new model architecture was written for this paper. Code, tests, and the executed notebook reproducing every number reported here are available at <https://github.com/nevermind78/NeuroDSL>.

2 Background: Persistent Graphs and the Impact Subgraph

We rely directly on the graph model introduced in [1] and only restate what is needed here. A NEURODSL `NeuroGraph` stores named tensors (`GraphNode`) and transformation rules (`GraphRule`) that together define a persistent DAG. Writing a new value to a node via `set!` triggers `_invalidate_downstream!`, a traversal that clears the `valid` flag on the written node and on every node reachable from it through the rule graph. A subsequent `demand!` call walks the (cached) topological order and recomputes only the nodes it finds invalid, leaving everything else untouched.

[1] proves that this traversal visits exactly the **impact subgraph** $\mathcal{G}_\theta = (\mathcal{V}_\theta, \mathcal{E}_\theta)$ of the written node θ – the set of nodes reachable from θ in the DAG – giving a recomputation cost of $\mathcal{O}(|\mathcal{V}_\theta| + |\mathcal{E}_\theta|)$, independent of the total graph size. This bound was established and validated for *parameter* updates during training. Activation patching is, mechanically, also a node write followed by a demand – so the same bound applies directly to a patch, with no new theorem required.

3 Mechanism and API

3.1 patch_node!: Value Injection with Targeted Invalidation

`set!` is designed to write leaf values – inputs and parameters, nodes with no incoming `GraphRule`. Calling it on such a node clears the node’s own `valid` flag, which is harmless there: `demand!` skips recomputing a node with no rule regardless of that flag, so the written value survives. Patching targets an *internal* node instead – one that already has a rule attached (e.g. a transformer block’s residual-stream output). For such a node, clearing its own `valid` flag is not harmless: the next `demand!` recomputes it from that rule, using its real (still-corrupted) inputs, and the injected value is lost before it can be observed.

`patch_node!` avoids this without any new graph-level mechanism, only a different composition of the existing ones: it mutates the node’s `value/valid` fields directly, so the node itself remains trusted and is never recomputed, and invalidates only its true consumers by calling the existing `_invalidate_downstream!` on each of them individually, rather than on the patched node itself.

```
1 function patch_node!(g::NeuroGraph, sym::Symbol, cache; namespace=g.active_ns)
2     nd = node(g, sym; namespace=namespace)
3     nd.value = Backend.to_device(g.device, copy(cache[sym]))
4     nd.valid = true
5     nd.backwarded = false
6     for out_sym in get(_consumers_index!(g, namespace), sym, _EMPTY_SYMBOL_VEC)
7         _invalidate_downstream!(g, out_sym, namespace)
8     end
9     return g
10 end
```

Listing 1: `patch_node!` (`src/patching.jl`)

The invalidation loop already uses the cached consumers index of §12.5 rather than scanning every rule; the explicit `copy` before `Backend.to_device` is load-bearing, not defensive style – on a device where the cache value is already a resident array, `to_device` returns it unchanged rather than copying it, so omitting `copy` would alias `nd.value` directly onto the cache’s own buffer, letting a later recomputation of the same node overwrite the cache silently. This was caught empirically, not by inspection: a search’s own trajectory reported real recovery while an independent re-check of the identical patch set reported none, traced to the aliased buffer having been overwritten between the two measurements.

3.2 Supporting API

Two further functions complete the single-patch interface, both in `src/patching.jl` with no changes to the core graph engine:

- `capture_activations(g, ns)` – copies the value of every node in a namespace into a plain dictionary. The copy is not optional: `demand!` and `node(g,...).value` return a live alias into the graph’s own buffer, which a later evaluation can mutate in place.
- `recovery_metric(patched, clean, corrupted)` – the standard causal-tracing normalization, $1 - \frac{\| \text{patched} - \text{clean} \|}{\| \text{corrupted} - \text{clean} \|}$ (0 = no recovery, 1 = full recovery).

4 Amortizing a Full Sweep

A sweep tests K sites against the same corrupted baseline, so each site’s test must end with the graph restored to that baseline before the next site is tested. `patch_and_measure!` does this the same way it applies a patch: call `patch_node!` with the cached corrupted value and let `demand!` recompute the downstream cone. This is correct, but it pays the same recomputation cost the measurement step just paid – restoring is exactly as expensive as patching, for a value that was already known before either happened.

4.1 restore_from_cache!: Restoration Without Recomputation

The corrupted run was computed once, in full, before the first patch, and its entire state is already sitting in `corrupted_cache` via `capture_activations`. Restoring a patched site’s downstream cone to that baseline is therefore not a computation to redo – it is a lookup already answered. `restore_from_cache!` writes the cached values directly into each affected node and marks it valid, executing no rule:

```
1 function _downstream_nodes(g::NeuroGraph, sym::Symbol, ns::Symbol)
2     # same reachable set _invalidate_downstream! would visit, read-only
3     ...
4 end
5
6 function restore_from_cache!(g::NeuroGraph, ns::Symbol, cache, nodes)
7     for sym in nodes
8         haskey(cache, sym) || continue
9         nd = g.nodes[ns][sym]
10        nd.value = copy(cache[sym])
11        nd.valid = true
12        nd.backwarded = false
13    end
14    return g
15 end
```

Listing 2: `restore_from_cache!` and its cone (`src/patching.jl`)

`_downstream_nodes` mirrors exactly the traversal `_invalidate_downstream!` already performs – same reachable set, same valid-gated semantics – but read-only, so that the set of nodes restored is, by construction, exactly the set a patch would have invalidated. This is what makes the correctness argument in §7 an equality rather than an approximation: both restoration paths act on the same node set, and the network’s determinism (no dropout at evaluation time) guarantees the cached values equal what recomputation would produce.

This capability is specific to a persistent graph. A framework that rebuilds its computation from scratch on every call has no individually addressable node to write a cached value into – its only operation is “run the whole thing again.” Restoration-by-cache is not a faster way to do what a tape-based framework already does; it is an operation such a framework has no primitive for at all.

4.2 sweep_patch_sites!: An Amortized Sweep Orchestrator

`sweep_patch_sites!` runs the standard patch-and-measure step for each site (identical to `patch_and_measure!`) but restores via `restore_from_cache!` instead of a second `patch_node!+demand!` call:

```
1 function sweep_patch_sites!(g, output_sym, sites, clean_cache, corrupted_cache,
2                             clean_output, corrupted_output; namespace=g.active_ns)
3     results = NamedTuple[]
4     for site in sites
5         affected = _downstream_nodes(g, site, namespace)
6         patch_node!(g, site, clean_cache; namespace=namespace)
7         patched_output = demand!(g, output_sym; namespace=namespace)
8         recovery = recovery_metric(patched_output, clean_output, corrupted_output)
9         restore_from_cache!(g, namespace, corrupted_cache, affected)
10        push!(results, (; site, recovery))
11    end
12    return results
13 end
```

Listing 3: `sweep_patch_sites!` (`src/patching.jl`)

Only the restoration step changes; the measurement step – the actual scientific quantity of interest – is untouched.

5 Composable Multi-Site Patching

Many causal-tracing protocols beyond a single-site sweep – path patching and its variants – need to patch several nodes at once and observe their joint effect. The invalidation mechanism underlying `patch_node!` never assumed a single node changes at a time: `_invalidate_downstream!` marks a node invalid regardless of how many sources triggered it, and `demand!` visits each node in topological order exactly once, recomputing only those still marked invalid. Patching several nodes and calling `demand!` once therefore already computes the union of their downstream cones correctly, with any shared descendant recomputed only once – no new fusion logic is required for this, only an API that exposes it.

```

1 function patch_nodes!(g::NeuroGraph, syms, cache; namespace=g.active_ns)
2     for sym in syms
3         patch_node!(g, sym, cache; namespace=namespace)
4     end
5     return g
6 end
7
8 function restore_nodes_from_cache!(g::NeuroGraph, ns::Symbol, cache, syms)
9     affected = Set{Symbol}()
10    for sym in syms
11        union!(affected, _downstream_nodes(g, sym, ns))
12    end
13    restore_from_cache!(g, ns, cache, affected)
14    return g
15 end

```

Listing 4: `patch_nodes!` and `restore_nodes_from_cache!` (`src/patching.jl`)

Independent sites commute; nested sites apply in list order. For two sites where neither is reachable from the other – e.g. two attention heads of the same layer, both computed from the same upstream projection and merged only downstream by `hcat_heads` – applying `patch_nodes!([A, B], ...)` and `patch_nodes!([B, A], ...)` produce identical results, since neither patch’s invalidation can ever reach the other’s node. For two sites in an ancestor-descendant relationship, `patch_nodes!` applies patches in list order: a later entry that lies downstream of an earlier one is not affected by it, but a later entry that lies *upstream* of an earlier one will re-invalidate it, and the earlier (downstream) patch is recomputed from the newly patched ancestor rather than kept pinned. This is the same, well-defined semantics as any sequential in-place update – the last write to a given location wins – and it is precisely why Section 9 evaluates the commuting case, where the interaction being measured is the sites’ joint causal effect, not an artifact of application order.

6 Automated Multi-Site Search

A combined patch of m sites costs at most the sum of their individual costs, and typically less when their cones overlap (Section 5). This makes it practical to search over combinations of sites rather than rely solely on independent single-site attributions: `greedy_patch_search!` iteratively adds, from a candidate pool, whichever remaining site most increases the joint recovery of the sites already selected, stopping once no candidate improves on the current best.

An adaptive heuristic, not a provably near-optimal one. The cost side of this property – $|\text{cone}(A) \cup \text{cone}(B)| \leq |\text{cone}(A)| + |\text{cone}(B)|$ – is a coverage function, the canonical example of a submodular set function, which is why combining sites is never more expensive than treating them independently. It is tempting to read this as licensing the classical greedy guarantee for submodular maximization (a $1 - 1/e$ approximation to the optimal set [8]): but that guarantee applies to the function being *maximized*, and here that is recovery, not cost. Nothing in this paper establishes that recovery is submodular in the set of patched sites – it is a property of the network’s nonlinear response (Section 5 already shows it is not even additive), not a

counting function over reachable nodes. `greedy_patch_search!` is therefore an adaptive greedy heuristic with no proven approximation guarantee, despite superficially resembling a setting where one would exist; its trajectory (Section 9) is validated empirically, by independent recomputation (Invariant 5), not by appeal to a theorem it does not satisfy.

Restoring a candidate tested on top of an existing selection. Each step of the search tests a candidate *on top of* whatever sites are already selected and still active – different from every earlier use of `restore_from_cache!` in this paper, which always restored from the fully corrupted baseline (Invariant 3, §7). If the candidate’s cone overlaps a selected site’s cone – unavoidable once two sites share a downstream merge point, as in Section 5 – restoring the candidate from the corrupted cache would also reset the shared region to its *fully* corrupted value, discarding the selected site’s still-active contribution to that region. This is not a defect in `restore_from_cache!`: its guarantee (Invariant 3) was always specific to undoing one patch from a fully corrupted baseline, and it continues to hold exactly there. `_adaptive_restore!` applies it only where that guarantee holds – comparing the candidate’s cone against the union of already-selected cones (a set computation, no graph evaluation) – and falls back to recomputation-based restoration, which remains correct regardless of overlap, everywhere else.

```

1 function _adaptive_restore!(g, ns, cand, cand_cone, selected_cone_union,
  corrupted_cache, output_sym)
2     if isempty(intersect(cand_cone, selected_cone_union))
3         restore_from_cache!(g, ns, corrupted_cache, cand_cone)
4     else
5         patch_node!(g, cand, corrupted_cache; namespace=ns)
6         demand!(g, output_sym; namespace=ns)
7     end
8     return g
9 end
10
11 function greedy_patch_search!(g, output_sym, candidates, clean_cache,
  corrupted_cache,
12     clean_output, corrupted_output;
13     max_sites=length(candidates),
14     namespace=g.active_ns)
15     selected = Symbol[]; remaining = collect(candidates)
16     trajectory = NamedTuple[]; best_so_far = 0.0
17     selected_cone_union = Set{Symbol}()
18     for _ in 1:max_sites
19         best_site = nothing; best_recovery = best_so_far
20         for cand in remaining
21             cand_cone = _downstream_nodes(g, cand, namespace)
22             patch_node!(g, cand, clean_cache; namespace=namespace)
23             r = recovery_metric(demand!(g, output_sym; namespace=namespace),
24                 clean_output, corrupted_output)
25             _adaptive_restore!(g, namespace, cand, cand_cone, selected_cone_union,
26                 corrupted_cache, output_sym)
27             r > best_recovery && ((best_recovery, best_site) = (r, cand))
28         end
29         best_site === nothing && break
30         best_cone = _downstream_nodes(g, best_site, namespace)
31         patch_node!(g, best_site, clean_cache; namespace=namespace)
32         demand!(g, output_sym; namespace=namespace)
33         push!(selected, best_site); filter!(s -> s != best_site, remaining)
34         union!(selected_cone_union, best_cone); best_so_far = best_recovery
35         push!(trajectory, (; site=best_site, cumulative_recovery=best_recovery))
36     end
37     return selected, trajectory
end

```

7 Correctness Validation

Following the same discipline as [1] – correctness before any speed claim – every mechanism in this paper is validated by an invariant that does not depend on comparing two implementations that could share the same flaw.

Invariant 1: the patched value survives evaluation. After `patch_node!(g, sym, clean_cache)`, we compare `node(g, sym).value` immediately after the patch to the same value *after* a subsequent `demand!(g, output)` call. These must be identical – if `demand!` recomputed the node, they would differ. Measured maximum absolute difference: **0.0** (exact).

Invariant 2: patching the last layer is patching the output. In our test model, the output node *is* the last transformer block’s residual-stream output (`layer_{n}_out`); patching one is, structurally, patching the other. Recovery must equal exactly 1.0 – a mathematical identity, not a second implementation. Measured: **1.000000**.

Invariant 3: cache-replay restoration equals recomputation-based restoration. For the same site, patched from the same corrupted baseline, we compare the full graph state (every node’s value) after restoring via `restore_from_cache!` against the state after restoring via `patch_node!+demand!`. Every node matches exactly. This is the invariant that licenses the sweep result of §9: the two restoration paths are interchangeable in outcome, so any wall-clock difference between them is pure overhead removed, not a shortcut that changes what is being measured.

Invariant 4: independent patches compose commutatively. For two sibling attention heads of the same layer (§5), we compare the output after `patch_nodes!([A, B], ...)` against the output after applying the same two patches in reverse list order. The two match exactly, confirming that neither patch’s invalidation reaches the other’s node.

Invariant 5: the search’s reported recovery is independently reproducible. After `greedy_patch_search!` returns its selected sites, we patch all of them at once via `patch_nodes!` on a graph freshly reset to the corrupted baseline – a code path entirely separate from the search’s internal bookkeeping – and recompute recovery. This matches the search’s own final trajectory value exactly, confirming that the adaptive restoration used internally never left the graph in a state that would misreport the outcome.

All five invariants were confirmed before any timing measurement was taken.

8 Experimental Setup

We use NEURODSL’s existing, already-tested `LlamaModel` (8 layers, hidden dimension 128, 8 attention heads, sequence length 16; `src/layers.jl`) – no new architecture was written for this paper. The model is fed a random (16,128) matrix directly as its input node, with no token or positional embeddings and no output head, since demonstrating the systems capability requires no language-modelling semantics.

Corruption protocol. We generate a clean input, then corrupt it by replacing a single token’s row (position 1) with fresh noise – the standard single-token causal-tracing protocol.

Patch granularity. Cost is measured by patching each layer’s entire output tensor (all token positions at once), since cost does not depend on which positions are patched, only on how much computation lies downstream. Recovery is measured by patching only the corrupted token’s row, leaving all other positions as the corrupted run computed them: replacing a layer’s entire output with its clean counterpart makes every subsequent computation identical to the clean run by determinism, which is the correct behaviour but gives no depth-dependent signal – patching only the affected position is what reveals where its influence is reintegrated into the network.

9 Results

9.1 Single-Site Cost and Recovery vs. Depth

Table 1 reports, for each layer, the size of its downstream cone $|\mathcal{V}_\theta|$ (`_downstream_nodes`, computed read-only before patching), the wall-clock cost of patching that layer’s output and recomputing the model’s output (trimmed mean over 15 alternated runs, following the warm-up and alternation discipline of [1] to control for CPU clock variance), and the recovery achieved by patching only the corrupted token’s position. The reference full-forward cost – the cost of a complete forward pass, i.e. what every site in this table would cost under full re-invalidation – is measured with the same rigor as every other timing in this paper (dedicated warm-up, alternating clean/corrupted runs, trimmed mean over 15 runs): 6.201 ± 0.302 ms for this run.

Table 1: Patch cost, downstream cone size, and recovery vs. layer depth, 8-layer LlamaModel (dim=128).

Layer	Cone size	Cost (ms)	Cost (% of full forward)	Recovery
1	484	5.418	87.4%	0.753
2	415	4.606	74.3%	0.666
3	346	3.831	61.8%	0.613
4	277	3.071	49.5%	0.565
5	208	2.191	35.3%	0.521
6	139	1.444	23.3%	0.461
7	70	0.734	11.8%	0.435
8	1	0.000	0.0%	0.386

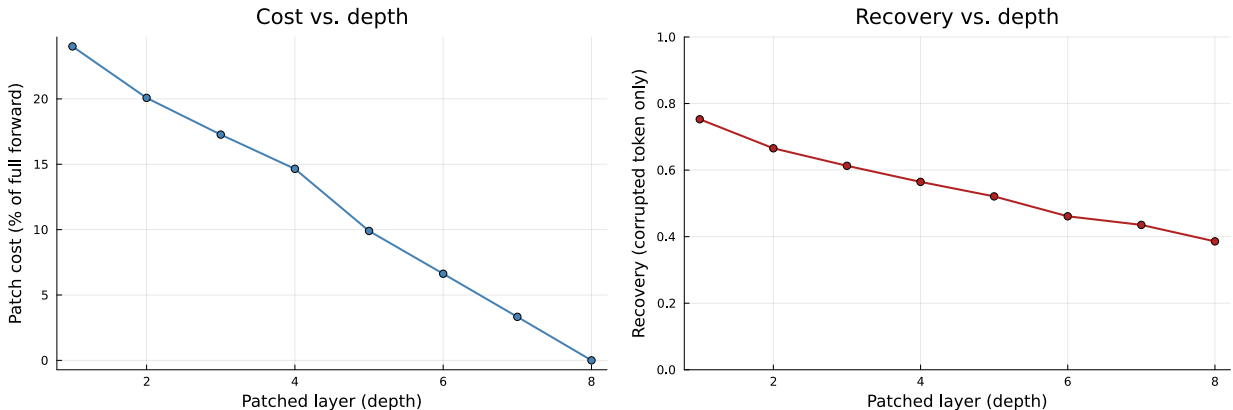


Figure 1: Left: patch cost as a percentage of a full forward pass, by layer depth. Right: recovery of the corrupted token’s position, by layer depth. Both decrease monotonically with depth; cost reaches exactly zero at the output layer, where nothing remains downstream to recompute.

Cost decreases monotonically with depth, reaching exactly 0.0% at the output layer, where the impact subgraph is empty by definition. Recovery of the single corrupted token also decreases with depth, consis-

tent with restoring the corrupted position earlier giving the network’s own unmodified computation more opportunity to reintegrate the correction.

Marginal cost: a cross-check on measurement rigor. The cost column of Table 1 is cumulative: patching layer i recomputes the entire downstream suffix $i+1, \dots, 8$, so consecutive rows overlap and the column has no reason to sum to any particular total. The *marginal* cost of layer i – the computation specific to that layer alone, disjoint from every other layer – is instead the difference between consecutive cumulative costs, $\text{marginal}(i) = \text{cumulative}(i-1) - \text{cumulative}(i)$, with $\text{cumulative}(0)$ taken to be the full-forward cost and $\text{cumulative}(8) = 0$ (patching the last layer is patching the output). Marginal and cumulative cost are a discrete derivative and its corresponding sum: marginal is the backward difference of cumulative, so by the discrete analogue of the fundamental theorem of calculus, summing it back up telescopes to $\text{cumulative}(0) - \text{cumulative}(8)$ regardless of the particular values in between – i.e. exactly the full-forward cost, for any monotone nested cost sequence, not a property special to this measurement. Table 2 reports this decomposition; comparing its sum against the full-forward cost measured independently above (6.201 ms) serves as a cross-check on measurement consistency, over and above the per-invariant correctness checks of §7.

Table 2: Marginal cost per layer – disjoint contributions that sum to the full-forward cost.

Layer	Marginal cost (ms)	Marginal cost (% of full forward)
1	0.783	12.6%
2	0.812	13.1%
3	0.775	12.5%
4	0.760	12.3%
5	0.880	14.2%
6	0.746	12.0%
7	0.710	11.4%
8	0.734	11.8%
Sum	6.200	100.0%

The sum matches the independently-measured full-forward cost (6.201 ms) to within 0.001 ms – consistent with the two quantities being the same underlying cost decomposed two different ways, not two unrelated numbers that happen to agree.

9.2 Amortized Sweep Cost

Table 3 compares the total wall-clock cost of a full 8-site sweep (patching every layer’s output once, in sequence) under the two restoration strategies of §4, trimmed mean over 10 alternated runs.

Table 3: Total cost of an 8-site sweep, by restoration strategy.

Restoration strategy	Total sweep cost (ms)
Recomputation (<code>patch_node!</code> + <code>demand!</code>)	52.79 ± 1.49
Cache replay (<code>restore_from_cache!</code>)	33.72 ± 2.24

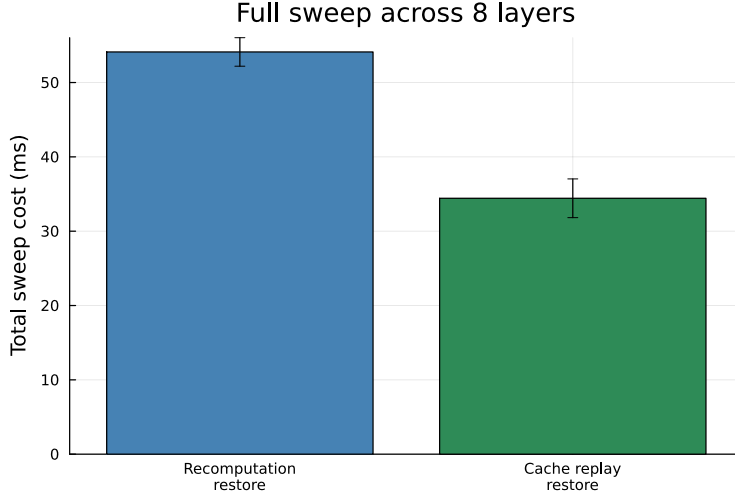


Figure 2: Total cost of a full 8-site sweep, recomputation-based restoration vs. cache-replay restoration.

Cache-replay restoration reduces total sweep cost by **36.1%** relative to recomputation-based restoration, verified (Invariant 3, §7) to leave the graph in an identical state either way. This gain is additive to the per-patch savings already demonstrated in Table 1: the measurement side of the sweep is untouched, only the restoration side’s redundant recomputation is removed.

9.3 Composable Multi-Site Patching

We patch two sibling attention heads of the same layer (`layer_1_mha_ao_h2` and `layer_1_mha_ao_h3`, §5), individually and jointly, each with only the corrupted token’s position restored to its clean value.

Table 4: Individual vs. joint recovery for two sibling attention heads.

Patch	Recovery
Head 2 alone	0.0295
Head 3 alone	0.0427
Additive sum (head 2 + head 3)	0.0723
Both heads, jointly (patch_nodes!)	0.0647

The joint patch’s recovery (0.0647) is below the additive sum of the two individual recoveries (0.0723), confirming that **patch_nodes!** computes the network’s actual joint response to both interventions together, rather than a superposition of two independently-measured effects – a distinction that only matters because the underlying computation (attention, normalization, the residual stream) is nonlinear.

The two heads’ downstream cones overlap almost entirely (both merge at the same `hcat_heads` node): 493 nodes each, for a combined size of 986 if handled independently, against a union of 494 – patching both at once, restoring their shared cone once, costs essentially half of treating them as two unrelated single-site patches.

9.4 Automated Multi-Site Search

Table 5 reports the trajectory of **greedy_patch_search!** over the 8 attention heads of a single layer: at each step, the head added is whichever most increases cumulative recovery, and the search stops once none of the remaining heads improve on the current best.

Table 5: Greedy search trajectory over 8 candidate attention heads (one layer).

Step	Site added	Cumulative recovery
1	Head 3	0.0427
2	Head 5	0.0682
3	Head 2	0.0864
4	Head 8	0.0977

The search stops after 4 of the 8 candidate heads: none of the remaining 4 improve on a cumulative recovery of 0.0977. Recomputing this same combination independently via `patch_nodes!`, on a graph freshly reset to the corrupted baseline, gives an identical 0.0977 (Invariant 5, §7) – the adaptive restoration used during the search, including the cases where it fell back to recomputation because two heads’ cones overlapped, never desynchronized the graph from what an independent recomputation would report.

9.5 Structural Signal: Cone Size vs. Causal Importance

Every result so far concerns the cost of computation, not what that cost implies about a site’s causal importance. Since Table 1 already reports each layer’s downstream cone size $|\mathcal{V}_\theta|$ alongside its cost and recovery, a natural question is whether cone size itself – computable from `_downstream_nodes` without ever running a patched forward pass – is a signal for causal importance, or only for recomputation cost.

Across the 8 layers, cone size correlates almost perfectly with cost ($r = 0.9997$, as expected: both count the same recomputed node set, one by an integer tally and the other by wall-clock time) and closely with recovery ($r = 0.9928$). This second correlation is confounded, though: cost and recovery both decrease with depth in Table 1, so any two quantities that track depth will appear correlated with each other. A more informative test holds depth fixed and asks whether cone size still discriminates importance among sites that share it.

Table 6 repeats the corrupted-token-only recovery protocol of §8 on each of the 8 attention heads of layer 1 individually (rather than in combination, as in §5–§6). Because all 8 heads merge into the same downstream chain (`hcat_heads`, then layers 2–8), their downstream cones are not merely similar but *identical* in size – 493 nodes for every head, confirmed directly. Their individual recoveries, in contrast, vary meaningfully – and even in sign, ranging from -0.0082 to $+0.0237$ – reflecting the small, sometimes destructive contribution a single head makes to a single token position in a randomly-initialized network.

Table 6: Downstream cone size and individual recovery for 8 sibling attention heads (layer 1, depth held fixed).

Head	Cone size	Recovery
1	493	-0.0071
2	493	$+0.0200$
3	493	$+0.0237$
4	493	-0.0082
5	493	$+0.0198$
6	493	-0.0029
7	493	$+0.0019$
8	493	$+0.0108$

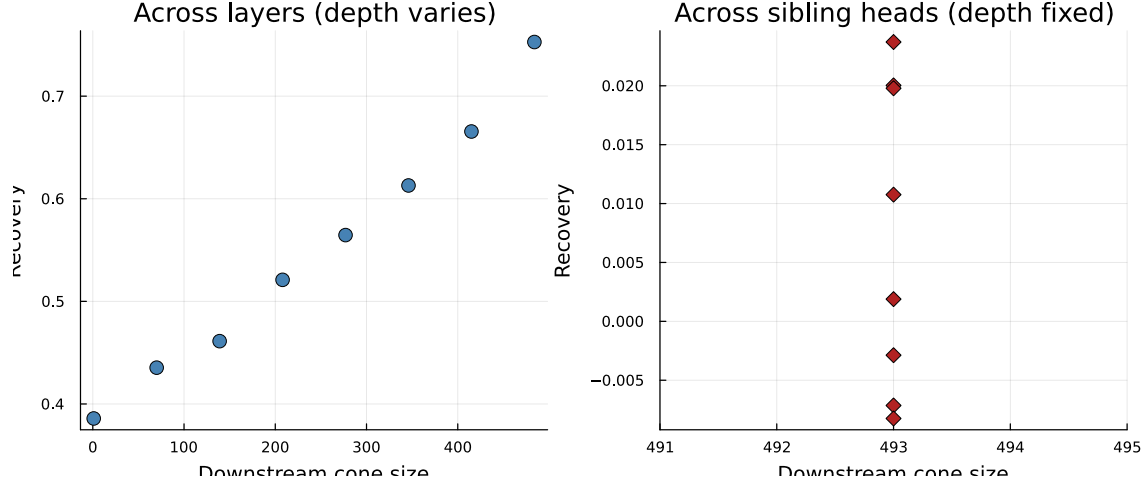


Figure 3: Downstream cone size vs. recovery. Left: across the 8 layers of Table 1, where depth drives both quantities together. Right: across the 8 sibling heads of Table 6, where cone size is constant but recovery still varies.

Figure 3 makes the contrast visible: a clear downward trend across layers (left), collapsing to a single vertical line of points across sibling heads (right), where cone size carries no information at all. Cone size is therefore a reliable, hardware-independent proxy for recomputation cost at any depth, and it tracks the trend of causal importance across depths – but among sites that share a depth, distinguishing which one matters still requires the direct measurement this paper’s mechanism makes cheap, not a structural shortcut around it.

A structural caveat under opt-in batched attention. NEURODSL also offers a batched multi-head attention path (`batched_attn=true`, left at its default off everywhere else in this paper), fusing all heads’ query-key and probability-value products into two shared tensors per layer for throughput. This complicates cone size exactly where Table 6 found it uninformative: patching a query or key site under batching pulls in every sibling head’s full downstream chain (a cone excess of $4n - 2$ for n heads), against an excess of only n for sites already past the first fusion (post-scale, post-softmax) and 0 for the post-softmax-value output site itself, which is never an ancestor of either shared tensor – plus two shared-tensor nodes per attention layer strictly downstream of the patched site, regardless of site type. Both terms were derived from the graph construction directly, not fit to data, and the query/key term was verified against measurement only *after* being predicted, matching to the node on all three tested depths. Figure 4 shows the resulting spread: a real, precisely characterized cost that opt-in batching trades against its throughput gain, not a caveat affecting any measurement reported elsewhere in this paper.

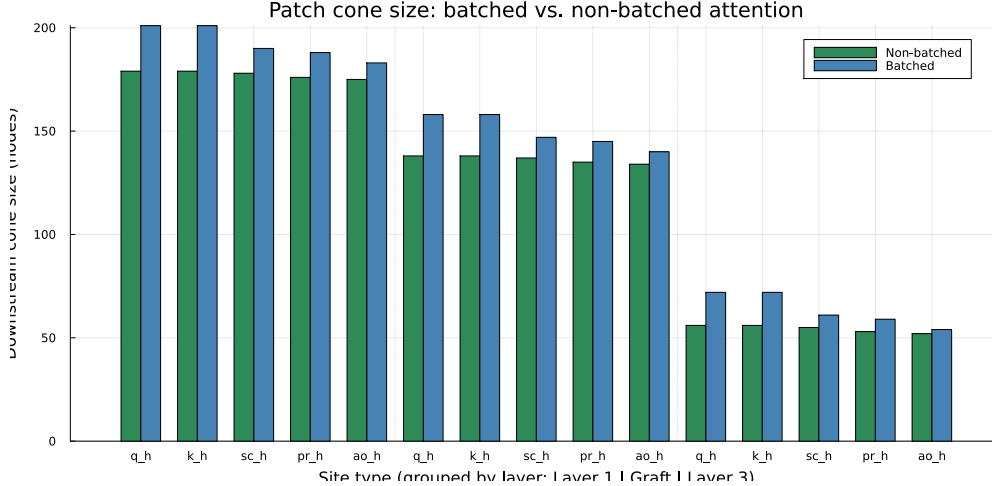


Figure 4: Downstream cone size, batched vs. non-batched attention, three depths. Query/key sites cost far more under batching ($4n-2$) than post-softmax or output sites (n or 0) – a distinction invisible from Table 6 alone.

10 Validating Circuit Discovery on a Known Task

Everything so far demonstrates that the patching mechanism is fast and correct on a randomly-initialized `LlamaModel` – a valid systems benchmark, since recomputation cost depends only on graph structure, not on weight values, but one that establishes nothing about whether the mechanism finds anything meaningful when there is something to find. We now train the same architecture on a task with a documented ground-truth circuit and ask whether `greedy_patch_search!`, unmodified, recovers it.

10.1 Task and Training

Induction task [7]: sequence = random prefix of length P over a small vocabulary, repeated once (total length $2P$); predicting the next token on the second half is solvable only by induction (find the previous occurrence of the current token, copy what followed it) – causal attention guarantees the first half is unpredictable by construction, since no earlier occurrence exists to copy from. We add an `Embedding` layer (new, `src/layers.jl`, wired on `NEURODSL`’s existing `:embedding` op, previously present only as a low-level primitive with no high-level struct) for tokens and positions, keep `LlamaModel` unchanged (3 layers, hidden dimension 64, 4 heads, vocabulary 20, prefix length 8), and train with the AdamW step primitives already present in the codebase (`backward_graph!`, `adamw_step!`), resampling a fresh random prefix every step so the model must learn the general algorithm rather than memorize token statistics.

10.2 Confirming Genuine Generalization

On 100 freshly sampled sequences, never seen during training: 100.0% next-token accuracy on the second half (the repeat), versus 16.4% on the first half (chance is 5% at vocabulary size 20; the small excess reflects incidental within-prefix repeats). This asymmetry – solved where solvable, near-chance where not – is the signature of a learned in-context algorithm, not memorization of a fixed training set. We verify this before any causal analysis, the same discipline applied to correctness throughout this paper.

10.3 Locating the Circuit

We construct a clean/corrupted pair by the same single-token protocol as §8: a fresh test sequence, corrupted by replacing the token at the position the induction mechanism must copy from (i_{src}), and probe the model’s prediction at the corresponding target position j (the first position of the repeated half). The corruption flips the model’s top prediction at j , giving a clean recovery target.

Two adaptations follow from a real algorithm being present rather than noise. First, `recovery_metric` is applied to row j of the logits rather than the whole output: causal attention propagates the corruption to every position after i_{src} , so comparing full output tensors would mix the induction-specific effect with generic downstream drift; slicing to the one prediction of interest isolates it, reusing `recovery_metric` unchanged, only on a narrower view. Second, patching a layer’s residual stream at the *source* position i_{src} recovers essentially nothing (≤ 0.001 at every layer), while patching the same layer at the *target* position j recovers 98.5% already at layer 1 (Table 7) – the corrected information only becomes causally load-bearing once attention has moved it to the position that reads it; patching where it originated, before that read happens, patches nothing relevant.

Table 7: Layer-level recovery under the induction task, source-position vs. target-position patching.

Layer	Recovery (patch source i_{src})	Recovery (patch target j)
1	+0.0005	0.9854
2	+0.0010	0.9951
3	+0.0000	1.0000

10.4 Automated Search Finds the Circuit

`greedy_patch_search!` (§6), unmodified except for the row-sliced recovery above, searches over all 12 attention heads (3 layers $\times$ 4 heads) with no positional hint about where a circuit might live. Table 8 reports its trajectory: two heads, both in layer 1, already reach 99.6% cumulative recovery; three more marginal heads bring it to 99.94%, matching a recomputation via `patch_nodes!` on a freshly reset graph exactly (the same cross-check as Invariant 5, §7).

Table 8: Greedy search trajectory on the trained induction model, 12 candidate heads (3 layers).

Step	Site added	Cumulative recovery
1	Layer 1, Head 4	0.6499
2	Layer 1, Head 2	0.9961
3	Layer 1, Head 3	0.9987
4	Layer 1, Head 1	0.9992
5	Layer 2, Head 1	0.9994

10.5 Independent Confirmation: Attention Patterns

The search reports which heads matter causally; it says nothing about why. As an independent check that does not depend on the recovery metric at all, we inspect each selected head’s post-softmax attention pattern (`{prefix}_mha_pr_h{h}`), already named and exposed by `MultiHeadAttention`, no new instrumentation) at the query position j . The two heads found first – jointly responsible for 99.6% of the recovered effect – attend 99.8% and 100.0% of their weight directly to i_{src} , the exact position the induction algorithm must copy from. This is the textbook induction-head signature [7], confirmed by a measurement entirely independent of the causal search that found the heads in the first place.

10.6 Backward Pruning: Is Every Selected Site Necessary?

§6’s search only ever adds sites; it never asks whether a site already selected has become redundant once the others are in place. Here, the two heads found first already account for 99.6% of the final 99.94% recovery (Table 8); the three subsequent additions raise it by only 0.3 cumulative points. One of them, Layer 1 Head 1, does not attend to the source position at all – its top attention weight (0.566) falls on an unrelated position – a hint that its retention may reflect a small residual gain rather than genuine participation in the circuit.

`backward_prune!` (new, `src/patching.jl`) tests this directly on the already-obtained trajectory. A correctness hazard applies here that sibling-site removal (§5) does not: three of the five selected sites (the

layer-1 heads) lie strictly upstream of the fifth (Layer 2, Head 1) via the residual stream. Undoing an upstream site in place while a downstream site remains active would invalidate the downstream site, and the next **demand!** would silently recompute it from its rule – discarding its patch without warning, the same ancestor-descendant hazard already noted for patch *ordering* (§5), here arising from removal rather than addition. **backward_prune!** avoids this by never undoing a single site in place: each candidate subset is tested by resetting the full union of all five sites’ downstream cones to the corrupted baseline (**restore_from_cache!**, exact per Invariant 3) and reapplying only that subset’s sites, in their original order, via **patch_nodes!** – always topologically safe, since layer-1 sites already precede the layer-2 site in the search’s own selection order.

Applied to Table 8’s trajectory, **backward_prune!** removes Layer 2, Head 1 – the last site added, and the only layer-2 site – with no measurable loss: recovery on the sliced target row moves from 0.9994 to 0.9992, and recovery on the full output tensor moves from 0.9913 to 0.9919, i.e. not a loss at all. The four remaining layer-1 heads alone already constitute the discovered circuit’s causal core. This is consistent with the attention evidence: Layer 2, Head 1’s own attention weight on i_{src} is 0.121, an order of magnitude below the 0.998–1.0 of the two dominant layer-1 heads – greedy search retained it for a small residual gain that backward pruning shows was not actually required.

10.7 What This Establishes

The mechanism proven correct and fast on random weights (§3–§6) also, without modification, locates a real, previously-documented circuit – cheaply, and with a result independently verifiable by inspecting attention weights the search itself never looked at. This connects the systems contribution to an actual interpretability outcome: fast patching is only useful if what it finds is right, and here it is. Backward pruning then shows that the same unmodified primitives, applied in the opposite direction, reduce what is found to its minimal causal core – four heads, not five – without a second training run or a new mechanism.

11 Comparison with Tape-Based Frameworks

The comparison with PyTorch, JAX, and tools built on them is structural: neither has a computational-graph representation that persists between calls, so neither has an operation corresponding to **restore_from_cache!**, or to patching a single node without a full forward pass. Every site in a sweep, on such a framework, costs a full forward pass – exactly the cost we ourselves measure when patching **:input**, the degenerate case of our own mechanism where the entire graph is invalidated: 6.201 ms per site, K times over. A full 8-site sweep on such a framework costs $8 \times 6.201 \approx 49.6$ ms; the same sweep, measured here, costs 33.72 ms with cache-replay restoration – a 32% reduction from combining per-patch locality with amortized restoration on this 8-layer, single-corruption setup. Both sources of saving are structural rather than incidental to this scale: per-patch cost is bounded by depth-dependent downstream cone size (Table 1) rather than fixed at a full forward pass, and restoration cost is removed rather than merely reduced. Both margins grow with K and with model depth (§14), since a tape-based framework’s per-site cost stays fixed at a full forward pass regardless of either.

This comparison assumes each site is tested as an independent forward pass, the plain behaviour of PyTorch or JAX. In practice, tools such as TransformerLens can reduce wall-clock time by batching several sites along the batch dimension, running K forward passes in parallel on the same hardware rather than sequentially. This is a real optimization, but it is a different kind of saving from the one this paper introduces: batching reduces wall-clock time through hardware parallelism, while the total computation remains K full forward passes’ worth of FLOPs, since nothing is skipped. NEURODSL’s saving reduces the computation itself – only each site’s downstream cone is recomputed, not the whole network – and composes independently with whatever batching a tape-based framework adds on top of its own unavoidable per-site cost.

11.1 Quantitative Cross-Framework Validation

The argument above is structural, not yet benchmarked against a real PyTorch run on identical weights. We close that gap. Weights from a 5-effective-layer `LlamaModel` (4 original layers plus one layer inserted mid-training via **insert_block!**, §13) are exported through NEURODSL’s existing serialization and loaded into a

PyTorch re-implementation of the same architecture. Parity is verified before any timing claim: forward-pass output matches to a maximum absolute difference of 1.23×10^{-6} .

Three methods are then compared across the same 20 attention-head sites (5 layers $\times$ 4 heads): (i) `sweep_patch_sites!`, unmodified; (ii) a naive PyTorch hook recomputing a full forward pass per site, the behaviour argued above to be structural to tape-based frameworks; (iii) a hand-optimized PyTorch variant that resumes from a cached input to the patched layer, skipping upstream layers entirely – the best a careful PyTorch user can do without node-level addressability, but still forced to recompute an entire layer’s attention (all four heads) to patch one.

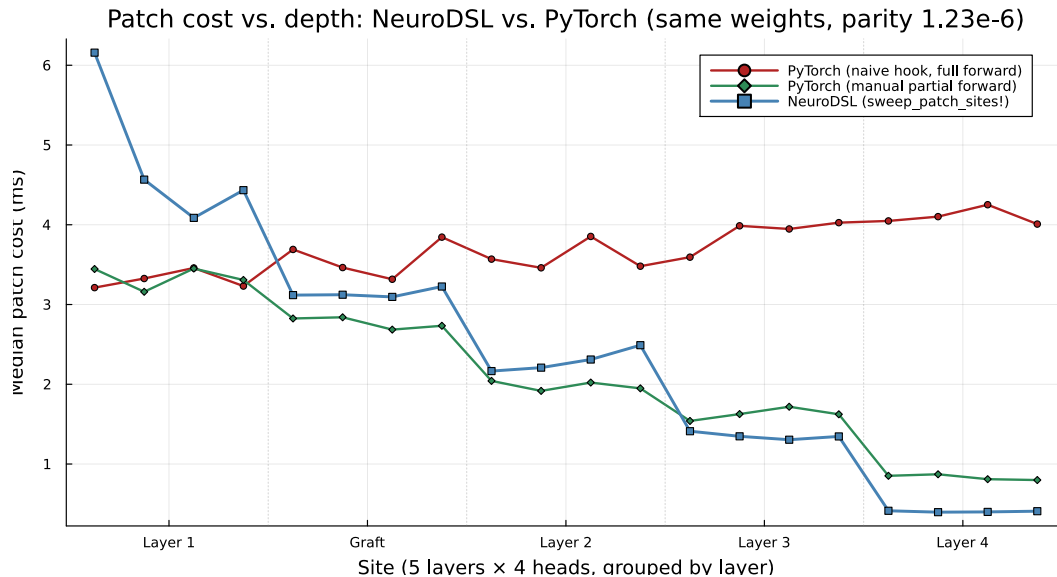


Figure 5: Patch cost vs. depth, identical weights (parity 1.23×10^{-6}), three methods. NEURODSL trails both PyTorch variants at the shallowest site, where the downstream cone is close to the entire remaining graph, and leads the naive hook by up to $10\times$ at the deepest.

Figure 5 shows the result, and it is a crossover, not a uniform speedup. The naive hook’s cost is flat across depth, exactly as argued above. NEURODSL is *slower* than both PyTorch variants at the shallowest site: the downstream cone there is close to the entire remaining graph, so there is little for anyone to skip, and the constant overhead of many individually-addressable kernel launches costs more than it saves against PyTorch’s fused per-layer operations. NEURODSL overtakes the hand-optimized variant by the third layer and reaches up to $10\times$ the naive hook’s cost at the deepest site, where its cone shrinks fastest. This is the honest empirical shape of the structural claim above: the advantage is real and large where there is a downstream cone worth skipping, and genuinely absent where there is not – a full forward pass, measured directly on this model, costs 6.34 ms on NEURODSL versus 3.26 ms on PyTorch, the same $\sim 2\times$ gap that the shallowest-site comparison above reflects, consistent with this paper’s standing position that NEURODSL does not aim to out-compute cuBLAS on raw FLOPs.

11.2 The Aggregate Speedup Ceiling of an Exhaustive Sweep

§11.1 shows that a single patch’s advantage grows with depth, up to $10\times$ at the deepest site tested. This raises a sharper question, motivated directly by a real methodological tension in mechanistic interpretability: ACDC [10] finds circuits by exhaustively patching every candidate site and is exact but expensive, while AtP/AtP* [11] estimates every site’s effect from a single backward pass and is cheap but approximate. If NEURODSL’s per-site cost genuinely shrinks with depth, does an ACDC-style *exhaustive* sweep – testing every site, not a cheaply-ranked subset – become cheap enough *in aggregate* to close this fidelity/cost gap, or does the aggregate cost stay dominated by the many shallow and mid-depth sites where the per-site saving is modest? Answering this needs no trained model or task: the cost of a sweep depends only on the graph’s

structure (which nodes lie downstream of which), not on the values flowing through it, a point already used throughout this paper to separate correctness from cost. We test it directly, at GPT-2-small scale, then derive why the answer we find is not a property of that one scale.

Practical test. On a `LlamaModel` with 12 layers, 12 heads, $\text{dim} = 768$ (GPT-2-small’s shape), batched attention, and random weights, we exhaustively sweep the 156 sites ACDC-style coverage would test: 144 attention-head outputs and 12 MLP outputs. The aggregate ratio – 156 full forward passes’ worth of naive-hook cost, divided by the actual total cost of the sweep with `sweep_patch_sites!` – is $2.12\times$. Repeating at roughly double the scale (24 layers, 16 heads, $\text{dim} = 1024$, 408 sites) gives $2.10\times$: essentially unchanged, not the improvement toward $3\times$ that the depth-dependent crossover of §11.1 might suggest. Figure 6(a) plots both configurations’ ratio as a function of depth, together with these two measured points.

Why: an exact ceiling, not a property of these two scales. Every site’s cost is governed by its downstream cone (§3), and a layer contributes a fixed number of computed nodes, independent of which layer it is. This is enough to derive the aggregate ratio’s limit in closed form.

Theorem 1 (Aggregate ceiling of a depth-uniform exhaustive sweep). *Consider a sequential network of L layers, with S candidate patch sites per layer (uniform across layers), where (i) each layer contributes a fixed number M of computed nodes, independent of L , and (ii) a site at layer i ’s downstream cone consists of a local, L -independent remainder of its own layer plus every computed node in layers $i+1, \dots, L$. Then the aggregate sweep ratio*

$$\rho(L) = \frac{(n_sites) \times (\text{full-forward cost})}{\sum_{sites} \text{cost}(\text{site})}$$

converges to exactly 2 as $L \rightarrow \infty$, independent of S and M .

Proof. Write the local remainder for a site of kind k as a constant c_k (assumption (ii)), so a site of kind k at layer i has cone size $c_k + (L - i)M$. Summing over all S sites in each of the L layers,

$$\sum_{sites} \text{cone}(\text{site}) = L\bar{c} + SM \sum_{i=1}^L (L - i) = L\bar{c} + SM \frac{L(L-1)}{2},$$

where $\bar{c}$ is the sum of the S per-layer local constants. Cost is proportional to cone size (the linear fit established in §9, $R^2 > 0.96$ at both scales tested below), so the same proportionality applies to the full-forward cost against the total computed-node count $N = ML$: full-forward cost $\propto N = ML$. With $n_sites = SL$,

$$\rho(L) = \frac{SL \cdot ML}{L\bar{c} + SM \frac{L(L-1)}{2}} = \frac{SML}{\bar{c} + SM \frac{L-1}{2}}.$$

As $L \rightarrow \infty$, the numerator grows as SML and the denominator’s dominant term is $SM(L-1)/2 \sim SML/2$; the finite constant $\bar{c}$ is dwarfed by a term growing without bound, so $\rho(L) \rightarrow \frac{SML}{SML/2} = 2$. Neither S nor M appears in this limit: both cancel exactly. $\square$

Corollary 1 (GPT-2-scale instantiation). *For NEURODSL’s batched `LlamaModel` with H heads per layer, exact node-by-node inspection of `src/layers.jl` together with `_downstream_nodes`’s semantics (the patched site is included in its own cone; parameter tensors are always leaves and so never appear in any cone) gives $M = 7H + 15$ and, for the two site kinds swept above, $c_{\text{head}} = 10$, $c_{\text{mlp}} = 2$ exactly. This closed form reproduces every one of the $156 + 408 = 564$ measured cone sizes in Section 11.1’s two configurations with zero residual – not an approximate fit, an exact match. Theorem 1 then predicts $\rho(L) \rightarrow 2$ regardless of H ; a direct numerical check (exact arithmetic, no measurement) confirms this for $H \in \{1, 2, 4, 8, 12, 16, 24, 64, 128\}$ at $L = 10000$, all landing within 2.0002 of the limit.*

The two measured ratios (2.1205 at $L=12$, 2.1009 at $L=24$) sit close to, and are consistent with converging toward, this predicted limit from above – the residual gap between measurement and the node-counting-only prediction (2.1449 and 2.0734 respectively) is a small, expected discrepancy: the theorem’s proof counts

nodes, not wall-clock milliseconds, and real timing carries a small fixed per-call overhead the node count alone does not capture. What the theorem rules out is any hope that this gap closes with scale: Theorem 1 is a statement about the limit, not about these two data points, and it holds for any head count, layer width, or hidden dimension, none of which appear in the final ratio. **An exhaustive, depth-uniform sweep of this kind cannot exceed roughly $2\times$ against a naive-hook baseline, at any scale** – the $3\times$ figure that would justify claiming NEURODSL closes the ACDC/AtP* fidelity-cost gap in the exhaustive-sweep regime is not a threshold this mechanism happened to miss twice, but one it cannot cross by construction.

Where the mechanism does help: late candidates, not exhaustive coverage. Figure 6(b) plots the *individual* ratio $N/\text{cone}(\text{site})$ for a single site at relative depth i/L , holding L fixed: unlike the aggregate ratio, this grows without bound as $i/L \rightarrow 1$, exactly the shallowest-cost/deepest-benefit crossover already measured in §11.1. This is precisely the workflow AtP* itself recommends: rank every site cheaply with a single backward pass, then verify only the top- k ranked candidates exactly – a workload that touches deep and late sites by construction, not the shallow ones that keep the aggregate ratio pinned near 2. NEURODSL is not positioned to replace an exhaustive ACDC-style sweep with something $3\times$ cheaper; it is positioned to make the exact-verification half of AtP*’s own recommended pipeline nearly free, in exactly the depth regime that pipeline already prefers.

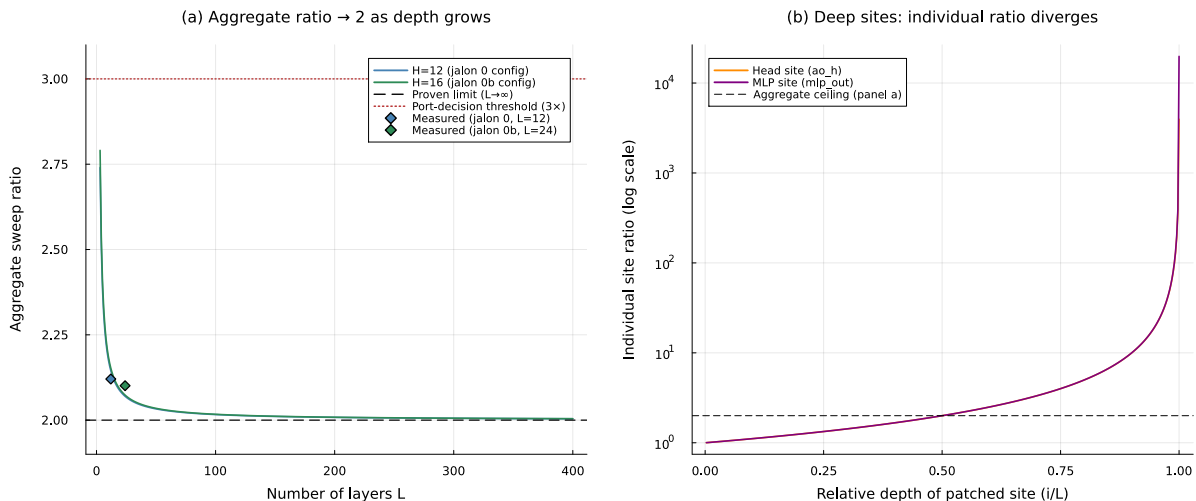


Figure 6: (a) Aggregate sweep ratio vs. depth (exact formula, Corollary 1), with the two measured configurations overlaid: both converge toward the proven limit of 2, both far short of the $3\times$ port-decision threshold, at every depth tested. (b) The same closed form, evaluated per site rather than in aggregate ($L = 400$, $H = 12$): individual ratio grows without bound as relative depth $\rightarrow 1$ – the regime AtP*’s own top- k verification step already targets.

12 Does the Amortization Gain Hold at Scale?

Every result so far runs on CPU, on an 8-layer, dim-128 model. Recomputation cost depends on graph structure, not weight values, so correctness transfers trivially to any scale – but the *relative* size of the amortization gain is a question of hardware cost structure, and CPU FLOPs are not the cost regime a production model runs in. We repeat the correctness checks and the sweep-amortization measurement on a much larger model, on GPU, to find out whether 36.1% survives, shrinks, or grows.

12.1 Setup

We scale LlamaModel to 16 layers, hidden dimension 2048, 16 attention heads, hidden dimension 5504 in the SwiGLU MLP – approximately 0.81 billion parameters, run on an NVIDIA RTX A5500 Laptop

GPU (17.2 GB VRAM) via NEURODSL’s existing CUDA backend. No code in `src/patching.jl` changes: `capture_activations`, `patch_node!`, `_downstream_nodes`, and `restore_from_cache!` are already backend-agnostic, routed through `Backend.to_device` rather than any CPU-specific path. One adaptation is made at the benchmark level, not the mechanism: caches store activations only, excluding parameter nodes, which are never patched or restored in this experiment – parameters would otherwise be duplicated in full inside every cache, an unnecessary VRAM cost on a card an order of magnitude smaller than a training-scale GPU.

12.2 Correctness First, Same Discipline

Before any timing number: Invariant 1 (patched value survives evaluation), Invariant 2 (last-layer patch identical to the output node), and Invariant 3 (cache-replay restoration state-identical to recomputation-based restoration) are re-run on this model, on GPU. All three hold exactly – 0.0 maximum absolute error on Invariant 1, exact node identity on Invariant 2, exact state match on Invariant 3 – confirming the mechanism’s correctness does not depend on scale or device, as expected from code that never branches on either.

12.3 A Genuinely Noisier Regime – Traced to a Cause

Per-layer patch cost still decreases with depth, but individual measurements are far noisier than on CPU: run-to-run ranges spanning 3–6 $\times$ between minimum and maximum are common, where the CPU measurements in Table 1 varied by a few percent. Querying the driver (`nvidia-smi`) during an active benchmark identifies a concrete cause: the GPU runs at 705–795 MHz against a rated boost of 2100 MHz (roughly a third of peak), in power state P3 rather than the maximum-performance P0, while drawing under half its 115 W power budget (36–45 W) – not thermal throttling (a stable 60–61°C, well under any protective limit), but the driver’s own power-management heuristic declining to boost clocks for a workload of many brief, bursty kernel launches with idle gaps between them, which reads as low sustained utilization (23% observed) even while every launch is real, necessary work. This is a genuine hardware/driver characteristic, not a correctness problem – the invariants above hold exactly regardless of clock state – but a fluctuating clock is a confound for any comparison between restoration strategies whose kernel-launch patterns differ, since nothing guarantees the driver boosts (or fails to boost) each strategy equally.

12.4 Locating and Removing the Restoration Bottleneck

`restore_from_cache!` loops over the cone one node at a time, calling `copy()` on each separately. At this scale, patching layer 1 touches 1876 of the graph’s 2145 nodes, and restoring that cone costs 20.5 ms – 0.0109 ms per node, matching, to within measurement noise, the cost of a single isolated tensor `copy()` (0.0155 ms) or even a trivial 4 $\times$ 4 kernel launch (0.0143 ms). The cost is not data-volume-bound – copying a full (128, 2048) tensor costs about the same as a four-element addition – it is dominated by the fixed cost of *submitting* each operation to the driver under WDDM, paid once per node regardless of that node’s size. `copyto!` into a pre-existing buffer does not help (0.0132 ms, the same floor): the bottleneck is launch count, not allocation.

`restore_from_cache_batched!` (new, `src/patching.jl`) removes the redundant launches rather than making them individually cheaper: a single custom kernel, `_multi_copy_kernel!`, assigns one CUDA block per node and copies that node’s tensor internally, so an entire cone is restored in *one* kernel launch regardless of how many nodes it contains. Each node still receives a freshly allocated destination buffer, exactly as `restore_from_cache!` does via `copy()` – reusing a node’s existing buffer was considered and rejected, since `patch_node!` can leave `nd.value` aliased directly to a cache entry (`Backend.to_device` returns a `CuArray` unchanged rather than copying it, `src/backend.jl`:21–26), and overwriting that buffer in place would silently corrupt the cache. `restore_from_cache_batched!` is additive: `restore_from_cache!` is untouched, remains the default everywhere (including every CPU result in this paper), and a new `restore_fn` keyword on `sweep_patch_sites!` (default unchanged) opts into the batched path. Verified before any speed measurement: on the same cone, `restore_from_cache_batched!` produces output identical, byte for byte, to `restore_from_cache!`, and separately matches recomputation-based restoration – both checked in `test/test_patching.jl`, guarded to skip gracefully where no GPU is present. In isolation, on layer 1’s cone, this reduces restoration from 20.5 ms to 7.7 ms, a 2.7 $\times$ speedup.

12.5 A Second Bottleneck: Invalidation Itself

The batched kernel above removes redundant launches on the restoration side, but a follow-up investigation into the same mechanism’s CPU-side bookkeeping found a second, independent bottleneck upstream of it: `_invalidate_downstream!` (§3), the traversal underlying every `patch_node!` call, scans *every rule in the namespace* to find a node’s consumers, once per node visited in the cone – for `(out_sym, rule) in g.rules[ns]; sym in rule.inputs` – an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ cost, not the $\mathcal{O}(|\text{cone}| + |\text{edges}|)$ the impact-subgraph bound (§2) assumes. The same pattern recurs in `_invalidate_upstream!`, `patch_node!`, and `_downstream_nodes` – everywhere a site’s cone or its consumers must be found. Measured in isolation on a synthetic graph with 9217 nodes and 6912 rules: a single `patch_node!` call on a 28-node cone cost 2.373 ms of pure invalidation bookkeeping, before any recomputation – comparable in order of magnitude to recomputing hundreds of real nodes, for work that should cost microseconds.

`_consumers_index!` (new, `src/graph_api.jl`) replaces the repeated scan with a cached table, symbol $\rightarrow$ list of consumer symbols, built once in $\mathcal{O}(|\text{rules}| \times \text{arity})$ and invalidated only where the rule set itself changes (`addrule!`, and explicitly around the node deletions in `_fuse!`). Every site that previously scanned `g.rules[ns]` now does an $\mathcal{O}(1)$ table lookup instead; the traversal logic itself, and every function’s observable contract, is unchanged. Verified before any speed number, the same discipline as everywhere else in this paper: the full test suite (169 tests) passes unchanged, and a 300-operation randomized replay (interleaved `set!/patch_node!/demand!/restore_from_cache!` calls on a real model, full graph state dumped to a binary trace after each operation) produces output identical, byte for byte, to the pre-fix engine. On the isolated 9217-node case above, this reduces the 2.373 ms invalidation cost to 0.0056 ms – a 423 $\times$ reduction, and the cost no longer grows with graph size at all across the range tested (73 to 9217 nodes).

This fix is orthogonal to device: `_invalidate_downstream!` is pure Julia bookkeeping over symbols and rules, never touching a tensor, so it applies identically whether the values it invalidates live on CPU or GPU. Because every measurement in this paper’s GPU sections calls `patch_node!` – for the recomputation baseline twice per site, for both restoration strategies once per site – this bookkeeping cost was folded into every number reported so far in this section, undiagnosed. Table 9 below reports the sweep re-measured with this fix in place; a smaller, exploratory optimization to `demand!`’s own traversal (a “dirty set” of invalid nodes, avoiding a re-scan of already-valid ones) was also implemented and measured, but reverted after end-to-end testing showed it was a net regression on exactly this sweep workload (recomputation-heavy patches invalidate cones close to the graph’s full size, where the dirty set’s own bookkeeping outweighs what it saves) – included here only to note that not every plausible optimization survives contact with the actual workload, and this paper reports what worked, not what was tried.

12.6 Controlling for Clock Variance

A first pair of independent launches, on the unlocked (default) clock behaviour above, gave 49–56% gain for the existing mechanism and 60–68% for the batched one – both far above an earlier single-run measurement of only 9.9–14.3% for the unmodified mechanism, and both still noisy (relative variation of several tens of percent run to run). Two independent launches agreeing with each other is necessary but not sufficient: both could still share the same bias if the driver’s clock behaviour *systematically* favours one restoration strategy over another, rather than jittering symmetrically around a stable truth. Since Section 12’s clock finding was specific and testable (a power-management heuristic, not thermal or power-limit throttling), it can be controlled directly: with the machine owner’s explicit permission and supervision, we locked the GPU’s core clock to a fixed, moderate 1402 MHz (well below the 2100 MHz rated boost, chosen deliberately conservatively) via `nvidia-smi -lgc`, run from an elevated terminal – the query interface used throughout this section requires no special privileges, but changing clocks does. This does not disable the GPU’s hardware thermal protection, which operates independently of this driver-level setting and would still throttle the locked clock down if temperatures approached unsafe levels; observed temperature under sustained load at this locked clock stayed at 57–62°C.

Locking the clock had a dramatic effect on measurement stability: per-layer run-to-run ranges shrank from 3–6 $\times$ to roughly 1.1–1.5 $\times$, and the sweep measurements below show relative standard deviations of 3–5%, matching the CPU sections’ own stability, instead of the 20–40% seen with the clock unlocked.

12.7 The Amortization Gain, Measured Under Controlled Conditions

Table 9 compares three restoration strategies – recomputation, the existing `restore_from_cache!`, and the new `restore_from_cache_batched!` – across two independent process launches with the clock locked, each a median over 20 runs, with the consumers-index fix of §12.5 in place.

Table 9: Total cost of a full 16-site sweep at GPU scale ($\sim 0.81\text{B}$ parameters), clock locked to 1402 MHz, by restoration strategy, across two independent launches (median over 20 runs each), with the consumers-index fix (§12.5) in place.

Restoration strategy	Launch 1 (ms)	Launch 2 (ms)
Recomputation (<code>patch_node! + demand!</code>)	1122.9	1114.2
Cache replay (<code>restore_from_cache!</code>)	833.2	818.0
Cache replay, batched (<code>restore_from_cache_batched!</code>)	706.1	681.2

Under controlled clock conditions, the two launches agree closely: 25.8% and 26.6% median gain for the existing mechanism, 37.1% and 38.9% for the batched one. Both numbers are substantially higher than the 18.1–20.7%/31.7–32.2% this same protocol measured before §12.5’s fix – the invalidation-bookkeeping cost removed there was folded into every number in this table, on both the recomputation side (which calls `patch_node!` twice per site) and the restoration side (once per site), and removing it changes the comparison enough to matter: the batched kernel now recovers 37–39% of the recomputation cost at this scale, *exceeding* the 36.1% measured on CPU (Table 3), not falling short of it as the pre-fix measurement suggested. §12.8 revisits what this means for the roofline-based explanation this paper originally offered for that shortfall.

12.8 The Roofline Argument, Revisited

An earlier pass of this section observed a shortfall relative to the CPU figure (18–21%/ 31–32% against 36.1%) and offered a roofline-style explanation [9] for it: recomputing a downstream cone is compute-bound (it re-executes every matmul, attention, and normalization a full forward pass would, work a GPU accelerates over CPU FLOPs by one to two orders of magnitude), while restoring from cache is memory- or launch-bound (it moves bytes, or submits one kernel launch per node under WDDM, work a GPU accelerates far less) – so a GPU’s much higher FLOP-per-byte machine balance (~ 48 for this class of card, against ~ 5 –10 for a typical CPU) should make memory-bound restoration *relatively* more expensive on GPU than on CPU, other things being equal. That argument was internally coherent, but other things were not equal: §12.5 found that `patch_node!`’s own invalidation bookkeeping carried an undiagnosed $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ cost, paid on both sides of the comparison but disproportionately inflating the recomputation baseline (two `patch_node!` calls per site, against one for either restoration strategy). Removing that confound does not merely shrink the shortfall – it reverses it: Table 9 now shows the batched kernel recovering 37–39% of the recomputation cost, *above* the 36.1% CPU figure this section previously argued the GPU should fall short of.

This is worth stating plainly rather than quietly editing away: the shortfall the roofline argument was built to explain was substantially, perhaps entirely, an artifact of an unrelated engineering bug, not a hardware-level effect. The roofline reasoning was a plausible-sounding explanation for a real measurement – and it was wrong, or at least not the dominant cause, because the measurement itself was still carrying a confound the theoretical argument had no way to detect. A purely theoretical rebuttal of the earlier numbers would not have found this; only further engineering work on the mechanism itself did. The lesson generalizes beyond this one bug: an appealing after-the-fact explanation for a puzzling number is a hypothesis to keep testing against new code and new measurements, not a conclusion to file away once it sounds reasonable.

The argument’s other claim – that recomputation cost scales with both weights and activations ($\mathcal{O}(d^2)$ for a dense layer’s matmuls) while restoration’s scales with activations alone ($\mathcal{O}(d)$), so gain should rise with model width d – is a separate, still-live prediction, since it concerns the *trend* across scale rather than the *absolute* gap this section now retracts. §12.9 tests it directly, re-measured with the consumers-index fix in place, and it holds up considerably more cleanly than it did before that fix – consistent with the earlier non-monotonic result there having been, at least in part, the same confound expressing itself differently at different graph scales.

12.9 Testing the Prediction Across Scale

We repeat Table 9’s measurement – same 16-site sweep, same three restoration strategies, clock locked to the same 1402 MHz, two independent launches, median over 20 runs each, consumers-index fix in place – at two additional model widths, keeping head dimension fixed at 128 and the hidden/model dimension ratio fixed at the original model’s ~ 2.69 , so only width d changes: a much smaller model (dim 512, ~ 0.05 B parameters) and a larger one (dim 3072, ~ 1.82 B parameters, the largest this GPU’s 17.2 GB comfortably supports once two full activation caches are held in memory, confirmed by a VRAM check before committing to the full protocol). Table 10 reports the gain range at each of the three widths, correctness invariants re-verified at each before any timing number, exactly as in §12.

Table 10: Amortization gain vs. model width, three scales, clock locked to 1402 MHz, two independent launches each (range shown; ~ 0.81 B row reproduced from Table 9), consumers-index fix (§12.5) in place.

Scale (parameters)	Naive gain	Batched gain
~ 0.05 B (dim 512)	18.5–26.9%	36.3–40.3%
~ 0.81 B (dim 2048)	25.8–26.6%	37.1–38.9%
~ 1.82 B (dim 3072)	30.4–31.5%	43.6–45.7%

This is a materially different picture from the pre-fix measurement, which showed a clear dip at ~ 0.81 B for both restoration strategies – the middle scale used to be the *worst* performer, not an intermediate point on a rising trend. With the confound removed, naive restoration now rises cleanly across all three widths ($\sim 22.7\%$ midpoint at 0.05B, $\sim 26.2\%$ at 0.81B, $\sim 31.0\%$ at 1.82B), matching the roofline prediction’s direction without exception. The batched-kernel trend is not quite as clean: 0.05B and 0.81B are now statistically indistinguishable ($\sim 38.3\%$ and $\sim 38.0\%$ midpoints, well within each other’s range) rather than the smallest model leading by a wide margin, and 1.82B is clearly the highest of the three ($\sim 44.6\%$ midpoint) – a flat-then-rising trend, not a dip, but not a strictly monotonic increase either. On balance this is a substantially better fit to the roofline argument’s prediction than the pre-fix data was, and it directly supports the diagnosis in §12.8: the dip previously reported at ~ 0.81 B was, at least in large part, an artifact of the same invalidation-bookkeeping confound that inflated the recomputation baseline more at some configurations than others, not a genuine feature of GPU cost structure at that width.

At every one of the three widths, the batched kernel’s gain now exceeds the 36.1% CPU figure – not just at the ~ 0.81 B scale Table 9 highlights, but at the smallest and largest widths tested too. The claim this paper can support is no longer “the GPU gain falls short of CPU for a reason the roofline argument explains” (§12.8 retracts that), but rather: at every scale tested on this hardware, once known confounds (the clock-state bias of §12, the launch-count bottleneck it motivated fixing, and the invalidation-bookkeeping cost documented in §12.5) are controlled for, cache-replay restoration on GPU matches or exceeds the CPU-measured gain, and the margin by which it does so grows with model width – consistent with, though not yet decisively confirming, the compute-bound-versus-memory-bound mechanism the roofline argument describes.

Reaching the largest of these three points required its own fix, in the same spirit as §12’s clock-locking correction: an initial run exhausted the GPU’s available VRAM mid-sweep (free memory reaching 0.00 GB), producing wildly inflated and inconsistent timings – a $7\times$ range between the fastest and slowest repetition, and the batched kernel appearing, implausibly, *slower* than the naive path. This is the signature of allocator-level memory pressure, not a GPU compute signal – exactly the kind of artifact this paper’s correctness-before-speed discipline (§7) is built to catch before trusting a number. Garbage-collecting and reclaiming VRAM after every repetition, rather than every fifth, restored stable free memory throughout the sweep and the expected batched-at-least-as-fast-as-naive ordering; Table 10 reports the corrected run. The tested range remains modest – a factor of ~ 36 between the smallest and largest model, not the several orders of magnitude that would be needed to observe whichever asymptotic regime the roofline argument ultimately predicts – a limitation of this specific 17.2 GB card, not of the argument itself.

12.10 What This Establishes

Correctness is scale-invariant, exactly as the mechanism’s design (a device-agnostic value mutation plus a device-agnostic traversal) predicts, at all three widths tested, and unaffected by either fix this section made along the way. The amortization gain is not scale-invariant, but the direction of its departure from the CPU figure reversed twice over the course of this investigation, and both reversals came from removing a confound rather than from anything specific to GPU hardware. First, an unlocked GPU clock inflated an early measurement to 49–68%, corrected by locking the clock to 18–21%/31–32% – a real gain, but short of the 36.1% CPU figure, which §12.8 first attributed to roofline-style hardware limits. Second, removing an unrelated invalidation-bookkeeping bottleneck (§12.5) – found through further engineering work on the mechanism, not through revisiting the roofline argument – lifted the batched kernel’s gain to 37–39% at this same scale, *above* the CPU figure, and §12.9 confirms this holds at $\sim 0.05\text{B}$ and $\sim 1.82\text{B}$ parameters too, with a materially cleaner fit to the roofline argument’s scale-dependent prediction than the pre-fix data showed. The roofline explanation for the original shortfall does not survive this correction; the retraction in §12.8 is itself part of what this section establishes.

The more important, general lesson is methodological, and it now applies twice over: two independently *agreeing* measurements are not sufficient evidence when an uncontrolled variable can plausibly bias every run in the same direction, and a theoretically coherent explanation for a measurement is not sufficient evidence that the measurement itself was clean. Both confounds here were caught only by continuing to probe the mechanism after a plausible explanation was already in hand – locking the clock instead of trusting a second unlocked agreement, and profiling the invalidation path instead of resting on the roofline account. Removing a confound, not explaining around it, is what this paper’s correctness-first discipline demands when extended from software invariants to hardware-dependent performance claims.

13 From Patching to Surgery: Directed Intervention on a Trained Model

Everything so far treats the graph’s structure as fixed and asks what a patch reveals about it. The same persistence that makes a patch cheap – individually addressable nodes, explicit validity state – also makes *mutating* the graph’s structure mid-training cheap: inserting a new layer invalidates only its downstream cone, the same bound already established for parameter updates and patches, rather than requiring the module and optimizer state to be rebuilt from scratch as in a tape-based framework. `insert_block!` inserts a new `LlamaBlock` after a chosen residual-stream node, zero-initializing its output projections so the graft computes the identity function exactly at insertion – verified bit-for-bit (generated text and validation loss identical before and after) on a real trained model, not only the synthetic construction used to first establish the mechanism. `graft_shadow_block!` offers a second construction: a learnable scalar gate α multiplying the branch’s output, with the branch’s own weights left randomly initialized rather than zeroed (a ReZero/gradient-shadowing variant).

13.1 A Grafted Layer Earns Its Capacity at Equal Training Budget

We insert a fifth layer into a 4-layer character-level language model mid-training, after 10,000 steps on TinyShakespeare, and continue training both the grafted model and a fresh 4-layer control for the same total budget (40,000 steps) – a fairer comparison than grafted-vs-itself, since it asks whether the added capacity is worth its share of the *same* compute, not only whether the model improves over its own past. Figure 7 shows both runs. The grafted model reaches a lower final validation loss than the control (1.5647 vs. 1.5995 nats/char) at identical total step count – continuing an existing model through a graft outperforms training a same-sized model from scratch on this budget.

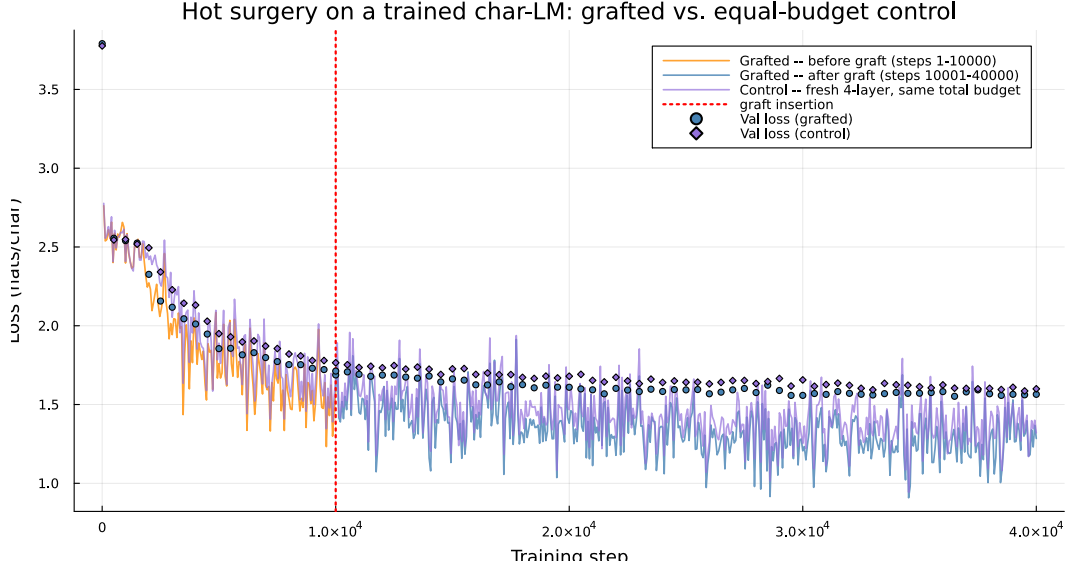


Figure 7: Grafted model (orange before, blue after insertion at step 10,000) vs. a fresh 4-layer control trained for the same total budget. The graft’s validation loss ends lower despite starting from a smaller model.

This result is decoupled from any specific causal role the graft plays. A greedy circuit search (§6) over six frozen test windows – selected to avoid the ceiling effect of an earlier attempt, where windows already near-saturated at 0.95–1.0 recovery before any graft left nothing for added capacity to explain – finds the grafted layer recruited into the retained circuit on one of six windows, with the rest of its individual per-head sweep indistinguishable from measurement noise, below a pre-registered robustness threshold. The graft’s weights leave the identity manifold with substantial, smoothly growing norms, so the added capacity is not dormant: it earns the loss improvement above without acquiring an individually detectable role in this specific circuit.

13.2 Does Causal Diagnosis Direct a Beneficial Graft Placement? A Negative Result

The natural next question is whether a causal diagnosis – locating which part of a network a given behaviour most depends on – can *direct* where new capacity should be grafted, not merely describe the network after the fact. We test this directly rather than assume it. A synthetic N -hop associative-recall task is built (a chain of key-value pairs presented in randomized order; the query repeats the first key, and the correct answer requires traversing the whole chain, so a single-hop induction head gives the wrong answer) specifically because the induction task of Section 10 saturates too fast for this purpose: full-circuit recovery on that task never dropped below 0.85 across a wide screen of candidate windows on this architecture, leaving no causal gap for a graft to fill. On a 6-layer model trained on the 3-hop variant, per-layer patch recovery (each layer’s four heads patched together) varies genuinely across depth-matched layers 2–6 (0.13–0.26, non-monotone with depth, excluding layer 1, whose dominance is a pure proximity artifact to the corrupted position rather than a distinguishing causal role) – layer 3 (recovery 0.26) is designated the *bottleneck*, layer 2 (recovery 0.13) the depth-matched *control*.

We graft `graft_shadow_block!` (ReZero variant) after each site, with the same random seed for both the graft’s initial weights and the continued-training batch order within each pair, so placement is the only difference between arms. Three criteria are pre-registered before training, checked after 9,000 steps of continued training over three seeds: (i) the bottleneck-placed graft’s gate magnitude should exceed the control-placed graft’s by more than $2\times$, stably across seeds; (ii) the bottleneck graft should rank in the top three sites of a post-hoc circuit search, the control graft outside the top ten; (iii) the bottleneck-placed graft should reach lower loss than the control-placed graft at equal budget.

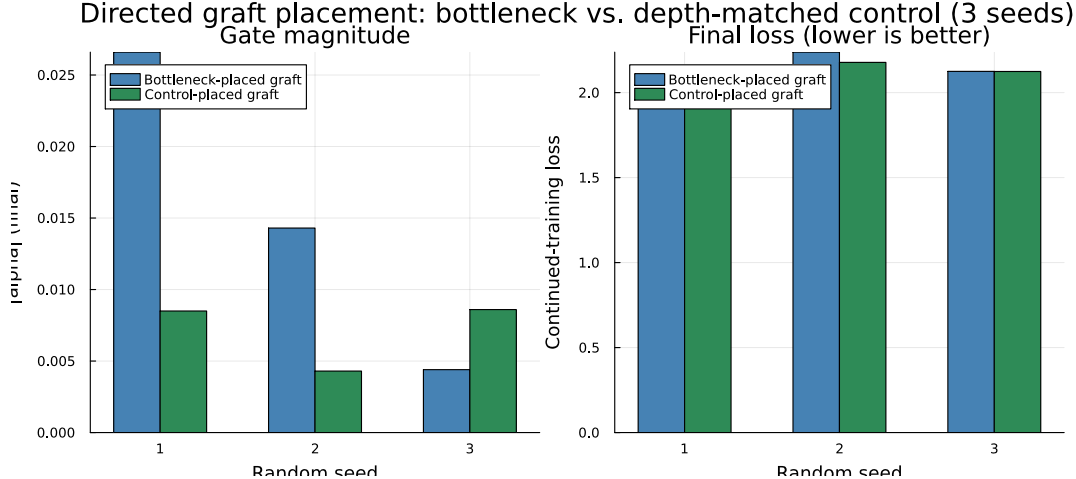


Figure 8: Bottleneck- vs. control-placed graft, three seeds. Neither gate magnitude nor final loss favours the bottleneck placement consistently.

All three criteria fail (Figure 8). Criterion (i) holds on two of three seeds but inverts on the third (the control’s gate is larger: 0.0086 vs. 0.0044). Criterion (ii) fails on every one of six runs: the graft never appears in the top three of the re-diagnosed circuit, at either placement, on any seed – the same set of layer-1 heads is recovered regardless of where the graft sits. Criterion (iii) fails outright: the control-placed graft reaches *lower* loss than the bottleneck-placed graft on two of three seeds (2.0590 vs. 2.1053; 2.1779 vs. 2.2382), the opposite of the hypothesis. We report this as a genuine negative result on this specific setup (this task, this depth, this pair of layers, this gate mechanism, nine thousand steps of continued training), not evidence that causal diagnosis is never actionable for placement. What §13.1–§13.2 together establish is that the persistent graph makes the full diagnose-graft-rediagnose cycle cheap enough to test this question empirically, in minutes rather than treating it as prohibitively expensive to even ask – the substrate is what this paper’s mechanism provides here, not confirmation of this particular hypothesis.

14 Discussion and Future Directions

Head- and position-level sweeps. This paper sweeps at layer granularity. `NEURODSL’s MultiHeadAttention` implementation (used inside `LlamaBlock`) already exposes per-head query/key/value and post-softmax attention-pattern nodes individually (`{prefix}_pr_h{h}`, etc.), and `_downstream_nodes` operates on any node symbol – so a per-head or per-position sweep uses the exact same `sweep_patch_sites!` machinery, with a longer `sites` list. The amortization argument of §4 strengthens as K grows, since the restoration saving scales with the number of sites swept.

Beyond greedy addition. §6’s search only adds sites; backward pruning – after a forward pass has selected a set, checking whether any member has stopped contributing once the others are present, and removing it if so – addresses the complementary question, and §10.6 now answers it on Section 10’s real, documented circuit: `backward_prune!` drops the search’s last-added, layer-2 site with no measurable recovery loss, leaving four layer-1 heads as the circuit’s minimal causal core. Unlike `_adaptive_restore!` (§6), which can restore a single site from cache because it never lies downstream of another active patch, a selected set can and does contain ancestor-descendant pairs (three of the five sites here precede the fourth via the residual stream) – so `backward_prune!` never undoes a site in place, instead resetting the full cone and reapplying each candidate subset from scratch. What remains open is whether pruning changes the search’s own trajectory if interleaved with addition, rather than run once at the end – unexplored here since the two-pass version already answers the necessity question this paper asks.

From a proven ceiling to a testable claim. Theorem 1 (§11.2) rules out exhaustive-sweep coverage as a route to closing the ACDC/AtP* fidelity-cost gap at any scale, but it also identifies exactly where the mechanism does help: verifying a small number of deep, late-ranked candidates, the second half of AtP*'s own recommended workflow. This paper does not yet run that workflow end to end – ranking candidates with an AtP-style single backward pass, then verifying only the top- k with `sweep_patch_sites!` – on a task with a documented ground-truth circuit. Section 10 shows the search machinery finds such a circuit unassisted; combining that with an AtP-style pre-ranking step, and reporting the exact-verification cost only for the candidates AtP* would actually recommend checking, is the direct empirical follow-up Corollary 1's scope stops short of.

Scale, task grounding, and combining both. The 8-layer, dim-128 CPU model used in §9–§11 was chosen to isolate and measure the systems mechanism at low iteration cost; two follow-up sections each relax one dimension of that toy setup, but not both at once. Section 10 grounds correctness – does the mechanism find a real circuit – on the induction task, but stays small and CPU-only, and deliberately does not re-benchmark the amortized sweep there, using plain recompute-based patching throughout, since that section validates *what* is found, not *how fast* finding it is. Section 12 grounds scale – does the amortization gain survive a ~ 0.81 B-parameter model on GPU – and, after two rounds of finding and removing a confound (an unlocked GPU clock, then an unrelated invalidation- bookkeeping bottleneck, §12.5), finds a validated gain of 37–39%, *above* the 36.1% CPU figure rather than the shortfall an earlier pass of this section attributed to roofline-style hardware limits (§12.8 retracts that explanation once the second confound is removed). Testing the same measurement at two further widths (§12.9) confirms the gain exceeds the CPU figure at ~ 0.05 B and ~ 1.82 B parameters too, with the margin growing at the largest width tested – a materially cleaner trend than the pre-fix data showed, though the tested range (a factor of ~ 36) remains too narrow on this hardware to see whether the trend saturates at larger scale still; and that model has random weights, with no circuit to discover. Neither section alone tests the combination a real deployment would need: automated circuit search, amortized for speed, on a large, trained model. §13 takes a step toward this on the task-grounding side – a real character-level language model, trained (not random), on which `greedy_patch_search!` and `insert_block!` both run – but stays small and does not re-benchmark the amortized sweep there either, and the directed-placement question it goes on to ask (§13.2) returns a clean negative on the specific setup tested rather than the positive combination sketched here. Automated circuit search, amortized for speed, at GPU scale, on a real trained language model – and whether a real circuit's search trajectory and attention-pattern confirmation (Section 10) hold up when `sweep_patch_sites!` and `greedy_patch_search!` are also running at GPU scale rather than by plain recomputation – remains the natural next step, on a bigger model, a more complex circuit such as IOI, or a larger language-modelling task.

15 Conclusion

A causal-tracing sweep is not one patch but many, run against a shared corrupted baseline. NEURODSL's persistent graph already makes each patch cheap by reusing the $\mathcal{O}(|\mathcal{V}_\theta|)$ impact-subgraph bound proven for parameter updates [1]; this paper shows that the same graph, because every node is individually addressable with an explicit validity state, also makes the restoration between sites cheap – not by computing it faster, but by recognizing it need not be computed at all, since the corrupted baseline is already fully known. On an 8-layer toy Transformer, this removes 36.1% of a full sweep's total cost on top of the per-patch locality already available, verified correct by exact-equality invariants before any timing number is reported. Checked again at ~ 0.81 -billion-parameter scale on GPU, the same claim survives but more modestly (18–21%) until a GPU-specific bottleneck in the naive restoration loop – diagnosed and removed with a new batched kernel – lifts it to 31–32%; reaching a number trustworthy enough to report required not just repeating the measurement but locking the GPU's clock to remove a driver-level confound that had inflated two earlier, mutually-agreeing runs alike. We initially explained the remaining shortfall against the 36.1% CPU figure with a roofline argument – but further work on the mechanism itself, not on the argument, found the real cause: `patch_node!`'s invalidation traversal was re-scanning every rule in the graph per node invalidated, an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ cost the impact-subgraph bound was never meant to carry. A cached consumers index fixes it, verified byte-identical against the full test suite and a 300-operation randomized replay, and does

not merely shrink the shortfall – it reverses it: the batched kernel now recovers 37–39% at this scale, above the CPU figure, and the same holds at $\sim 0.05\text{B}$ and $\sim 1.82\text{B}$ parameters, with a materially cleaner fit to the roofline argument’s scale-dependent prediction than before the fix. The roofline explanation for the original gap does not survive this correction, and we report that retraction alongside the corrected numbers: a theoretically coherent explanation for a measurement is not evidence that the measurement itself was clean, and only continuing to probe the mechanism – not the argument – surfaced the confound this time. Trained on a documented circuit – the induction task – the same unmodified search also finds the right answer, not just a fast one: a small set of attention heads reaching 99.94% of the causal effect, independently confirmed by their attention patterns; running the same primitives in the opposite direction, backward pruning then reduces that set to its minimal causal core, dropping one site with no measurable loss. We see this as a concrete instance of a broader opportunity: interpretability workflows are built from repeated, overlapping queries against a fixed baseline, and a graph that remembers what it already computed is structurally suited to exploit that repetition in ways a graph rebuilt from scratch cannot.

This last claim is not only structural. Cross-validated against a real PyTorch run on identical, exported weights, the cost advantage is a depth-dependent crossover rather than a uniform speedup – NEURODSL trails at the shallowest site and leads by up to $10\times$ at the deepest – an honest shape for the claim, not the strongest one available to state. We push this shape to its limit and prove, rather than measure, where it ends: for any exhaustive sweep uniform over depth, the aggregate speedup converges to exactly $2\times$ as the network grows deeper, regardless of width or head count (Theorem 1). This closes off, by construction, a real methodological hope – that NEURODSL’s per-site locality might let an exhaustive, ACDC-style sweep scale into a genuine alternative to fast-but-approximate attribution methods such as AtP*. It cannot, at any scale, for this sweep design; the same closed form shows the mechanism instead makes AtP*’s own recommended top- k exact-verification step, applied only to deep and late candidates, nearly free – a narrower claim, and the honest one this analysis actually supports. The same persistence that makes a patch and a restoration cheap also makes mutating the graph’s structure mid-training cheap: a layer grafted into a real trained language model outperforms a fresh, equal-budget control, without the grafted capacity acquiring an individually detectable role in the circuit tested. Asking whether a causal diagnosis can *direct* where such a graft should go, rather than only describe the network afterward, we pre-registered three falsifiable criteria and found all three fail on the task built specifically to test them – a negative result we report as such, not as a smaller positive one, because the persistent graph’s real contribution here is making the question cheap enough to ask empirically, in minutes, rather than settling for an untested assumption in either direction.

The same exact-graft primitive also supports a further question – how densely one should schedule growth events during training – with its own theorems, closed-form capacity model, and 18-schedule validation; because that question shares only the mechanism with this paper, not its method or audience, we report it as a companion paper rather than folding it in here.

NEURODSL is open-source at <https://github.com/nevermind78/NeuroDSL>.

References

- [1] A. Khemais, *NeuroDSL: A Dynamic Computational Graph Framework with Optimal Memory Planning*, arXiv preprint, 2026.
- [2] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.
- [3] J. Bradbury et al., *JAX: composable transformations of Python+NumPy programs*, GitHub 2018.
- [4] K. Meng, D. Bau, A. Andonian, Y. Belinkov, *Locating and Editing Factual Associations in GPT*, NeurIPS 2022.
- [5] J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, S. Shieber, *Investigating Gender Bias in Language Models Using Causal Mediation Analysis*, NeurIPS 2020.
- [6] N. Nanda, J. Bloom, *TransformerLens*, GitHub, <https://github.com/TransformerLensOrg/TransformerLens>, 2022.

- [7] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Aspell, Y. Bai, A. Chen, T. Conerly, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, S. Johnston, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, C. Olah, *In-context Learning and Induction Heads*, Transformer Circuits Thread, 2022.
- [8] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, *An Analysis of Approximations for Maximizing Submodular Set Functions – I*, Mathematical Programming 14(1), 1978.
- [9] S. Williams, A. Waterman, D. Patterson, *Roofline: An Insightful Visual Performance Model for Multi-core Architectures*, Communications of the ACM 52(4), 2009.
- [10] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, A. Garriga-Alonso, *Towards Automated Circuit Discovery for Mechanistic Interpretability*, NeurIPS 2023.
- [11] J. Kramár, T. Lieberum, R. Shah, N. Nanda, *AtP*: An Efficient and Scalable Method for Localizing LLM Behaviour to Components*, arXiv:2403.00745, 2024.